KernelLeakGuard

User Manual

Hybrid static analysis and runtime verification of memory leaks on Linux kernel early-exit paths

BITE303P Operating Systems Lab, Fall 2026-27, VIT Vellore

Om Gunjan Gupta (24BIT0316) · Abhishek Kumar Singh (24BIT0309) · Aarav Jain (24BIT0023)

Guide: Dr. Arivuselvan K

CHAPTER 1: INTRODUCTION

KernelLeakGuard finds memory leaks on the error paths of Linux kernel modules and checks each finding by reproducing the failure inside a virtual machine. This manual explains how to install it, run it, read its results and analyse your own module.

What you get for every finding:

- the function, the allocation (file and line) and the leaking return with its error code;
- the call whose failure leads to the leak (for example a second kmalloc() or copy_from_user());
- a verdict from the virtual machine: **confirmed** (kmemleak saw the leak), **not reproduced** (the failure was injected and nothing leaked) or **inconclusive** (the failure could not be injected);
- the evidence: fault number, stack frame, kmemleak backtrace.

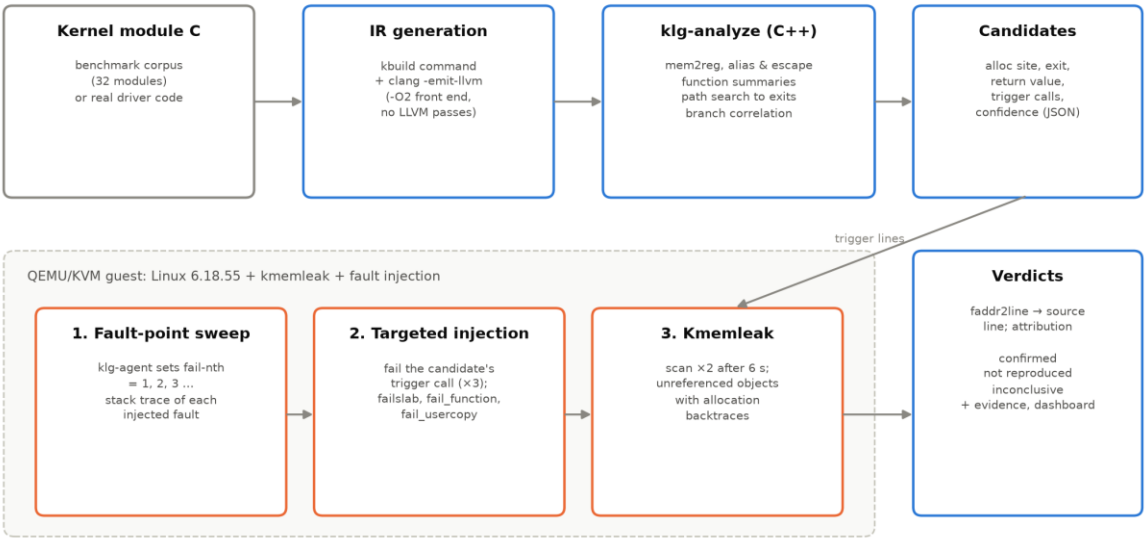


Figure 1.1: How the pieces fit together

CHAPTER 2: SYSTEM REQUIREMENTS

Table 2.1: Requirements

Item	Requirement
Operating system	Windows 10/11 with WSL2 (Ubuntu 24.04 or newer), or native Linux x86-64
Virtualisation	KVM available (/dev/kvm); on Windows 11 WSL2 provides it
CPU / RAM	4+ cores recommended, 8 GB RAM (the guest VM uses 2 GB)
Disk	about 25 GB (kernel source and build)
Network	only for the first download of packages and kernel source

CHAPTER 3: INSTALLATION

3.1 Install WSL2 and Ubuntu (Windows only)

Open PowerShell as administrator and run `wsl --install -d Ubuntu`, restart, then open the Ubuntu app from the Start menu and choose a user name and password.

3.2 Install packages

In the Ubuntu terminal:

```
sudo apt update
sudo apt install -y build-essential clang llvm llvm-dev lld libclang-dev \
  qemu-system-x86 flex bison bc libelf-dev libssl-dev dwarves cpio busybox-static \
  python3-venv python3-pip git curl xz-utils rsync kmod
sudo usermod -aG kvm $USER
```

Close the terminal and run `wsl --shutdown` in Windows PowerShell so the group change takes effect. Check with `ls -l /dev/kvm` and `id` (the kvm group must be listed).

3.3 Get the project and build the components

```
cd ~
git clone <repository-url> KernelLeakGuard      # or copy the project folder here
cd KernelLeakGuard
./vm/build_kernel.sh        # downloads and builds Linux 6.18.55 (10-20 min the first time)
./vm/build_initramfs.sh     # guest root filesystem with klg-agent
make -C analyzer            # static analyser -> analyzer/build/klg-analyze
./scripts/setup_venv.sh     # Python packages for dashboard and documents
```

Keep the project inside the Linux file system (for example /home/<user>), not under /mnt/c: the kernel build needs a case-sensitive file system and runs much faster there. From Windows the folder is reachable at \\wsl$\Ubuntu\home\<user>\KernelLeakGuard.

CHAPTER 4: RUNNING KERNELLEAKGUARD

4.1 The complete pipeline

```
python3 -m klg all
```

This builds the 32 benchmark modules and their LLVM IR, runs the analyser in six configurations, boots the VM twice (fault-point sweep, then confirmation) and prints the evaluation. It takes about 3-4 minutes. Expected output (numbers may vary slightly):

```
[klg] built 32 modules and their LLVM IR in 23.2s
[klg] static (full): 19 warnings in 32 modules
...
[klg] runtime verification: {'confirmed': 15, 'inconclusive': 3, 'not-reproduced': 1}
| Approach | TP | FP | FN | Precision | Recall | F1 |
| static/full | 16 | 1 | 1 | 0.94 | 0.94 | 0.94 |
| hybrid      | 15 | 0 | 2 | 1.00 | 0.88 | 0.94 |
| klg/combined | 16 | 0 | 1 | 1.00 | 0.94 | 0.97 |
```

4.2 Individual steps

Table 4.1: Command reference

Command	What it does
python3 -m klg build	build corpus modules, LLVM bitcode, klg-agent and initramfs
python3 -m klg analyze [--all-variants]	static analysis (full, or also the ablation variants)
python3 -m klg verify [--skip-sweep]	runtime verification in the VM (reuse the last sweep with --skip-sweep)
python3 -m klg evaluate	precision/recall against the ground truth, triage table
python3 -m klg show	one line per finding with its verdict and reason
python3 -m klg fix	generate fix patches for confirmed leaks and validate them (compile, re-analyse, VM re-run); patches in results/fixes/patches
python3 -m klg real [--only m1,m2]	analyse the real Linux modules of vm/real_modules.fragment and verify their initialisation in the VM (results/real)
python3 -m klg seed [--only name] [--skip-vm]	reintroduce a known leak into a real kernel module (seeds/) and run detect -> confirm -> fix on it (results/seeded); --skip-vm re-checks only the static part and keeps earlier VM results
python3 scripts/make_real_triage.py	write the manual-review labels of the real-module warnings (results/real/triage.json)
python3 scripts/make_root_causes.py	summarise the root-cause probe runs of the real-code findings (results/real/root_causes.json)
.venv/bin/python -m klg.figures	regenerate the charts in docs/figures
scripts/repeat_runs.sh N	run the pipeline N times to check reproducibility

A seed is a JSON file in seeds/ with the fields name, module_file (path in the kernel tree), function, remove (the release statement to comment out), occurrence, bug and reachable (descriptions for the report). Optional fields: module (the kernel module name when it is built from several files, e.g. null_blk from main.c), sweep_max (map only the first N fault points of module initialisation) and vm: false with vm_note (skip the VM for a seed whose leak kmemleak cannot observe, with the reason).

Reproducing the real-kernel findings. The helpers below re-run the evidence behind Section 4.4 of the report (results/real/root_causes.json). A fault-probe run boots the guest, loads one module with fail-nth set to each value in a range, and stores the kmemleak report and kernel log per batch.

Table 4.2: Real-kernel finding helpers

Command	What it does
python3 scripts/live_demo.py	narrated live demo in the guest kernel: the SLUB leak (erofs, fault 85) and the historical bonding leak, reported by kmemleak on screen (about 2 minutes without KVM)
python3 scripts/live_demo.py shell	root shell in the guest kernel (modules in /mnt/host/mods; leave with poweroff -f)
python3 scripts/live_demo.py --window (or: shell --window)	the same, on the virtual machine's own screen: a QEMU window on the desktop shows the kernel booting and every command, like a VirtualBox VM; the terminal mirrors the demo
python3 scripts/fault_probe.py erofs 85 85 1 OUT	fail the 85th fault point of erofs initialisation: SLUB leaks a slab cache (kmemleak)
bash scripts/build_patched_kernel.sh patches/0001-*.patch ~/klg-build/kbuild-slubfix	build a guest kernel with the SLUB fix in its own build directory
PROBE_KBUILD=~/klg-build/kbuild-slubfix python3 scripts/fault_probe.py erofs 85 85 1 OUT	the same fault point on the patched kernel: nothing leaks
PROBE_KO=path/to/f2fs.ko python3 scripts/fault_probe.py f2fs 369 429 61 OUT	probe another build of a module (e.g. f2fs with patches/0003)
python3 scripts/validate_patch.py ntfs3 fs/ntfs3 fslog.c PATCHED_FILE OUT	build a module with a patched source file and compare the static warnings
python3 scripts/make_root_causes.py	summarise all of the above into root_causes.json

4.3 The dashboard

```
.venv/bin/streamlit run dashboard/app.py
```

Open http://localhost:8501 in a Windows browser. Tabs:

- **Overview:** counts of warnings and verdicts, static vs hybrid precision, pipeline summary.
- **Findings:** filterable table of all findings. Selecting one shows the source with the allocation (blue), the failing call (orange) and the leaking exit (red), the suggested fix, and the runtime tests with kmemleak backtraces.
- **Fix patches:** the generated patches with their compile, re-analysis and VM status.
- **Real kernel code:** the 39 real modules: warnings with review notes, leaks found by the sweep, root causes and patches of the kernel bugs found, and the re-introduced leaks.
- **Live kernel:** boots the guest kernel and streams its console into the page. 'Run live demo' shows the SLUB leak and the historical bonding leak being caught by kmemleak; 'Kernel shell' gives a root shell in the guest (type a command and press Enter). With 'Also show the VM's own screen' ticked, the virtual machine's display opens as a window on the laptop at the same time.
- **Evaluation:** comparison table and precision/recall chart; triage of warnings.
- **Run pipeline:** runs any step and streams its output into the page.

CHAPTER 5: READING THE RESULTS

Table 5.1: Verdicts

Verdict	Meaning	What to do
confirmed	The predicted failure was injected and kmemleak reported an object allocated at the reported line as unreferenced.	Fix the bug; the evidence shows how to reproduce it.
not reproduced	The failure was injected but nothing leaked; the static warning is most likely a false positive (e.g. a correct rollback loop).	Review briefly; usually no action.
inconclusive	No injectable failure leads to the path, or the failure cannot happen in the test state.	Review the warning manually.

Table 5.2: Fields of a static finding (results/static/full/<module>.json)

Field	Meaning
alloc	file and line of the allocation that leaks
exit / return_value	the return statement on the leaking path and the value it returns
triggers	calls whose failure leads onto the path (the last one is the primary)
confidence	high: the function frees the object elsewhere but not on this error path; medium: ownership is transferred elsewhere, or never released; low: other cases
category	missing-release-on-error-path, ownership-not-transferred-on-error-path, never-released, missing-release-on-other-path

Output files: results/static/<variant>/*.json (findings), results/runtime/verification.json (verdicts and evidence), results/runtime/sweep|confirm/out/ (raw agent logs, kmemleak reports, guest console), results/evaluation.json (metrics).

CHAPTER 6: ANALYSING YOUR OWN MODULE

1. Put the module source into a directory with a Kbuild file (`obj-m += mymod.o`) and build it against the guest kernel:

```
make -C ~/klg-build/linux-6.18.55 O=$HOME/klg-build/kbuild LLVM=1 M=$PWD modules
```

2. Generate bitcode and analyse it:

```
python3 -c "from klg import build; from pathlib import Path;
print(build.emit_ir(Path('mymod.o').resolve(), Path('.')))"
analyzer/build/klg-analyze mymod.bc -o mymod.json
python3 -m json.tool mymod.json | less
```

3. For runtime verification, copy the module into corpus/ (or another directory passed to the verifier). Code that runs in module_init is verified automatically (init mode). For other code, expose the routine through a debugfs file with the harness macros in corpus/klg_harness.h (`KLG_TRIGGER(fn)`), then run `python3 -m klg all`.

4. Unknown allocators or release functions can be added without recompiling via `klg-analyze --spec extra.json`, for example `{"allocators": ["my_alloc"], "deallocators": {"my_free": 0}}`.

CHAPTER 7: TROUBLESHOOTING

Table 7.1: Troubleshooting

Problem	Solution
/dev/kvm not accessible	Add yourself to the kvm group (sudo usermod -aG kvm $USER) and restart WSL; without KVM the VM falls back to slow emulation.
Module build: unresolved symbols	Run ./vm/build_kernel.sh first (it creates Module.symvers).
VM run failed	Read results/runtime/<run>/console.log and out/job.log; a kernel panic message there shows what went wrong.
All verdicts inconclusive	Check that scripts/faddr2line works: ~/klg-build/linux-6.18.55/scripts/faddr2line corpus/<m>.ko <func>+0x10
Commands fail from Windows PowerShell	Run them inside the Ubuntu terminal, or with wsl -d Ubuntu --cd ~/KernelLeakGuard -- <command>.
Dashboard not reachable	Use http://localhost:8501; if the port is busy add --server.port 8502.